

SamGoG: A Sampling-Based Graph-of-Graphs Framework for Imbalanced Graph Classification

Shangyou Wang[†]
School of Artificial Intelligence and Data Science, University of Science and Technology of China
Hefei, Anhui, China
Data Darkness Lab, Suzhou Institute for Advanced Research, University of Science and Technology of China
Suzhou, Jiangsu, China
wash_you@mail.ustc.edu.cn

Zezhong Ding[†]
School of Artificial Intelligence and Data Science, University of Science and Technology of China
Hefei, Anhui, China
Data Darkness Lab, Suzhou Institute for Advanced Research, University of Science and Technology of China
Suzhou, Jiangsu, China
zezhongding@mail.ustc.edu.cn

Xike Xie[*]
School of Biomedical Engineering, University of Science and Technology of China
Suzhou, Jiangsu, China
Data Darkness Lab, Suzhou Institute for Advanced Research, University of Science and Technology of China
Suzhou, Jiangsu, China
xkxie@ustc.edu.cn

Abstract

Graph Neural Networks (GNNs) have shown remarkable success in graph classification tasks by capturing both structural and feature-based representations. However, real-world graphs often exhibit two critical forms of imbalance: class imbalance and graph size imbalance. These imbalances can bias the learning process and degrade model performance. Existing methods typically address only one type of imbalance or incur high computational costs. In this work, we propose SamGoG, a **sam**pling-based *Graph-of-Graphs* (**GoG**) learning framework that effectively mitigates both class and graph size imbalance. SamGoG constructs multiple GoGs through an efficient importance-based sampling mechanism and trains on them sequentially. This sampling mechanism incorporates the learnable pairwise similarity and adaptive GoG node degree to enhance edge homophily, thus improving downstream model quality. SamGoG can seamlessly integrate with various downstream GNNs, enabling their efficient adaptation for graph classification tasks. Extensive experiments on benchmark datasets demonstrate that SamGoG achieves state-of-the-art performance with up to a **15.66%** accuracy improvement with **6.7**× training acceleration.

CCS Concepts

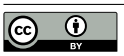
• **Computing methodologies** → **Neural networks**; *Supervised learning by classification.*

Keywords

Graph Neural Networks; Imbalanced Graph Classification; Scalability

ACM Reference Format:
Shangyou Wang[†], Zezhong Ding[†], and Xike Xie[*]. 2026. SamGoG: A Sampling-Based Graph-of-Graphs Framework for Imbalanced Graph Classification. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026), August 9–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817908

[†] Equal contribution.
[*] Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD 2026, Jeju Island, Republic of Korea.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817908

Resource Availability:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.20416232.

1 Introduction

Graph classification is a supervised learning task that seeks to learn a predictive function $f : g \rightarrow C$ on a set of graphs $\mathbb{G} = \{g_i\}_{i=1}^{N}$, utilizing a labeled subset $\mathbb{G}_{\mathrm{L}} = \{g_i\}_{i=1}^{N_{\mathrm{L}}} \subset \mathbb{G}$. Each graph $g_i \in \mathbb{G}_{\mathrm{L}}$ is represented as $g_i = (A_i, X_i, y_i)$, where A_i is the corresponding adjacency matrix, X_i is the node feature matrix, and $y_i \in C$ is the class label, with C being the set of possible classes. The goal is to learn a function f that generalizes to predict labels for unlabeled graphs in $\mathbb{G}_U = \mathbb{G} \setminus \mathbb{G}_L$, by leveraging their structures and features. This task finds broad applications in social network analysis [13] and bioinformatics [1, 17], with GNNs [18, 37] emerging as the state-of-the-art due to their capability to model structural and feature information jointly.

Despite the success of GNNs, graph classification under imbalanced conditions remains a significant challenge, primarily due to two common issues: **class imbalance**, where some classes contain far more training graphs than others [22, 28], and **graph size imbalance**, where input graphs vary substantially in size (i.e., number of nodes) [20, 49]. As illustrated in Figure 1 (a), class imbalance biases models toward majority classes, degrading the performance of minority ones, while graph size imbalance impairs performance on small graphs and hinders generalization under distribution shifts. Existing methods fall into data-centric (e.g., class resampling, synthetic augmentation) [5, 9, 42, 45] and algorithmic-centric (e.g., subgraph matching, pattern extraction) [20, 36, 49] categories, but both face computational bottlenecks and limited effectiveness in handling imbalance. Recent advances highlight the potential of the Graph-of-Graphs (**GoG**) paradigm [41] (Figure 1 (b)), which represents each input graph as a GoG node in a higher-level graph, with edges between GoG nodes defined via pairwise similarity between corresponding input graphs. This inter-graph representation enables richer supervision: by embedding minority-class or small

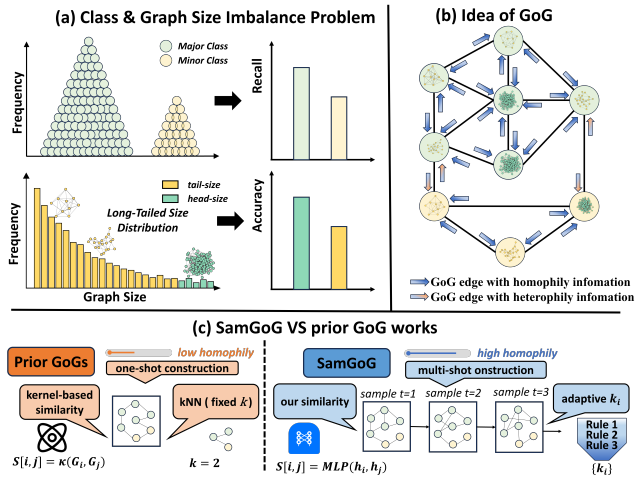


Figure 1: (a) In terms of Class Imbalance, major classes have high frequency and high recall, whereas minor classes have low frequency and low recall. Regarding Size Imbalance, tail graphs (small size) are numerous but yield low accuracy, while head graphs (large size) are scarce but yield high accuracy. (b) GoG models relationships between graphs by connecting with edges that capture both homophily (similarity) and heterophily (dissimilarity) information. (c) Compared with prior GoG works, SamGoG achieves high efficiency and effectiveness (high homophily) by using learnable similarity and multi-shot sampling-based construction instead of kernel-based similarity and one-shot construction.

graphs within a broader context of similar graphs, GoG-based methods enhance representation learning beyond what is possible using only intra-graph features.

However, existing GoG approaches [41, 45] encounter three major challenges that prevent them from fully harnessing the potential of GoG: **Challenge 1:** As edge homophily (see in Definition 1) is widely recognized as a crucial prerequisite for the effectiveness of GNNs [53], existing GoG methods treat edge homophily purely as a post hoc evaluation metric rather than incorporating it as an optimization objective during construction so that the resulting GoGs lack theoretical guarantees and capability to capture inter-graph supervision. **Challenge 2:** The construction of GoG edges relies heavily on pairwise graph similarity, which is typically computed using graph kernels (e.g., Weisfeiler-Lehman subtree [34] and shortest path [4]) that prioritize structural features over semantic ones. This struggles to precisely represent semantic relationships between graphs and incurs high computational cost (e.g., $O(N^2\overline{|V|}^4)$, where N is the number of input graphs and $\overline{|V|}$ is the average number of nodes per graph. **Challenge 3:** Existing methods adopt a one-shot kNN strategy for GoG construction, assigning a fixed number of neighbors k to each GoG node. However, this fixed k makes the resulting GoG's edge homophily highly sensitive to the choice of k[1]. Moreover, the one-shot construction introduces inductive biases and prevents the model from dynamically refining inter-graph relationships, underscoring the need for a more flexible and iterative GoG construction process.

To address these challenges, we propose **SamGoG**,a homophily-driven framework, designed to efficiently construct multiple GoGs by importance-sampling in the presence of class and graph size imbalance and trains on them sequentially, as illustrated in Figure 1 (c). **For Challenge 1**, we formulate GoG construction as a constrained optimization problem where node degrees are adaptively assigned (Section 3.1). We prove that prioritizing neighbors with better similarity quality can increase expected edge homophily. **For Challenge 2**, we investigate a learnable pairwise similarity function that jointly encodes structural and semantic features. This not only eliminates the overhead of kernel-based methods but also improves edge homophily by guiding the sampler to favor same-class neighbors. **For Challenge 3**, we propose: a) an adaptive GoG node degree allocation strategy that adjusts neighborhood size for each GoG node to achieve higher homophily under class and graph size imbalance (Section 3.1); and b) a dynamic and parallel sampling method that generates multiple GoGs to capture a wider range of graph-level interactions during training. This replaces one-shot kNN construction, reduces inductive bias, and allows inter-graph supervision (Section 3.3) to evolve throughout training.

Our contributions can be summarized as follows. **First**, we propose SamGoG, the first framework to formulate GoG construction as a constrained homophily maximization problem, theoretically establishing the relationship between node degree allocation and homophily maximization. Our analysis demonstrates that degree distributions prioritizing nodes with better quality of similarity enhance homophily stability under Lemma 1, providing a theoretical foundation for topology-adaptive sampling aligned with the class and graph size imbalances. **Second**, we design three complementary mechanisms for GoG construction: 1) a learnable pairwise graph similarity function replacing graph kernels, which serves as the importance metric for sampling, 2) an adaptive GoG node degree allocation strategy replacing fixed node degree in the kNN method, and 3) a dynamic sampling method replacing one-shot kNN construction to generate multiple GoGs during training. Compared to baselines, ours improves edge homophily by over 5%, while achieving a 4-6 order of magnitude reduction in similarity computation time. **Third**, SamGoG achieves state-of-the-art (SOTA) performance on imbalanced graph classification benchmarks [33], demonstrating significant improvements across diverse datasets. Specifically, it outperforms baselines with an average accuracy increase of 5.07% in 24 class imbalance experiments across 4 datasets, and an average accuracy gain of 4.13% in 24 graph size imbalance experiments on the same datasets. Moreover, SamGoG achieves up to a 6.7× acceleration over baselines. These results underscore SamGoG's outstanding performance in both accuracy and efficiency.

2 Preliminary

2.1 Graph-of-Graphs

The GoG represents a set of input graphs as nodes of a higher-order graph, therefore converting the problem of *graph classification* (assigning labels to input graphs) into a *semi-supervised node classification* task (assigning labels to GoG nodes).

[1]When applying G^{2}GNN [41] with the shortest path kernel to the PTC-MR [2] and DHFR [35] datasets (containing 344 and 756 graphs, respectively), increasing k from 1 to 10 leads to a notable drop in edge homophily—by 9% on PTC-MR and 15% on DHFR.

Formally, given a set of N input graphs $\mathbb{G} = \{g_i\}_{i=1}^{N}$, let $\mathbb{G}_{\mathrm{L}}$ and $\mathbb{G}_{\mathrm{U}} = \mathbb{G} \setminus \mathbb{G}_L$ denote the labeled and unlabeled subsets, respectively. Each input graph $g_i = (A_i, X_i, y_i) \in \mathbb{G}$ has s_i nodes (i.e., as graph size). The GoG is constructed as a high-order graph $G = (V, E, H, Y)$, where: (1) each GoG node $v_i \in V$ corresponds to an input graph $g_i \in \mathbb{G}$; (2) the GoG node feature $h_i \in H$ is encoded as $h_i = \mathrm{Readout}(\mathcal{M}(A_i, X_i))$, where $\mathcal{M}(\cdot)$ denotes a GNN for node encoding and $\mathrm{Readout}(\cdot)$ (e.g., mean/sum pooling [11, 44]) aggregates node embeddings; (3) GoG edges E capture relationships between input graphs (e.g., similarity); and (4) labels of GoG nodes $Y = \{y_i\}_{i=1}^{N}$ correspond directly to the labels of the original input graphs. Only the labels associated with the labeled subset $\{y_j \mid g_j \in \mathbb{G}_L\}$ are observable, while the labels $\{y_k \mid g_k \in \mathbb{G}_U\}$ remain unobserved and are the targets to be predicted.

The core of GoG construction lies in building the edge set E based on a precomputed pairwise graph similarity matrix $S \in \mathbb{R}^{N \times N}$. This process involves two key steps: 1) **Neighborhood Selection**: determine the count of edge connections k_i for each GoG node v_i, defining its neighborhood size. 2) **Edge Materialization**: for each v_i, update edge set E by adding directed edges from v_i to its k_i most similar counterparts in the GoG, as determined by the pairwise similarity matrix S. A comprehensive notation table summarizing all key symbols used throughout the paper is provided in Appendix Section A.

2.2 Related Works

GoG extends the conventional GNN paradigm [27]. Early works like NoN [14] and GoGNN [38] introduce GoG for graph classification and link prediction, respectively. G^{2}GNN [41] advances this idea by constructing a kNN-based GoG using graph similarity, alleviating class imbalance in graph classification via inter-graph message passing. Building on G^{2}GNN, ImbGNN [45] introduces a similarity-aware graph random walk to extract core subgraphs, enhancing the reliability of kernel-based similarity under graph size imbalance. This enhancement addresses G^{2}GNN's limitation in underestimating similarity between graphs of different sizes, but results in poorer efficiency due to the cost of extracting core subgraphs and extra computation in pairwise similarity. However, these prior GoG approaches face fundamental limitations, as described in Section 1.

3 SamGoG

Framework. This section introduces the SamGoG, a sampling-based GoG framework for imbalanced graph classification by optimizing edge homophily, as shown in Figure 2. Initially, SamGoG calculates the GoG node degree (k_i) for all training graphs based on the three rules investigated in Section 3.2.2. Then, SamGoG embeds the input graphs with GNN encoders to obtain graph representations, i.e., GoG node embeddings. Next, a multi-layer perceptron computes a pairwise graph similarity matrix $S \in \mathbb{R}^{N \times N}$ between graph representations. We then perform parallel sampling from an implicit complete graph[2], guided by the node degrees $\{k_i\}$ and the similarity matrix S. Each GoG node's degree determines how many neighbors it selects in the sampled sub-GoG, while S defines the connection probabilities between node pairs. This process generates multiple sub-GoGs. Finally, these sampled sub-GoGs are used for training downstream node classification models.

The rest of this section proceeds as follows. §3.1 formulates the GoG construction optimization objective based on edge homophily. §3.2 presents the methods of pairwise graph similarity computation and GoG node degree allocation. §3.3 analyzes subgraph training effectiveness on sampled GoGs and the complexity of several key components in the framework.

3.1 Graph-of-Graphs Construction Objective

We begin by defining the objective of GoG construction. Edge homophily is a widely acknowledged metric for evaluating the quality of GoG construction in existing works [41, 45]. Motivated by this, our goal is to maximize **edge homophily** [53] of constructed GoGs. It is worth noting that homophily can be defined in multiple ways [19, 29, 53]. To avoid ambiguity, we use "**homophily**" to specifically refer to edge homophily throughout this paper. Homophily is defined as follows:

Definition 1. ***(Homophily [53])**: Given a GoG graph $G = (V, E, H, Y)$, the homophily of G (homo(G))is defined as the fraction of edges linking two GoG nodes with the same label. Formally, we have homophily:*

$$\mathrm{homo}(G) = \frac{1}{|E|} \sum_{(v_i, v_j) \in E} \mathbb{I}(y_i = y_j) \tag{1}$$

, where $|E|$ denotes the number of edges. $\mathbb{I}(\cdot)$ denotes the indicator function that returns 1 when $y_i = y_j$, and 0 otherwise.

Although many recent studies [24, 25, 50] have empirically demonstrated that improved node classification performance generally but not always align with homophily, and other works explore specific methods under heterophily [3, 7, 53] or seek a general solution towards both homophily and heterophily [10, 46, 52], mainstream message-passing GNNs still implicitly assume edge homophily. Therefore, optimizing homophily remains a critical objective in constructing GoG. Additionally, homophily allows tail-size and minor-class GoG nodes to enhance their representations through neighborhood aggregation, thereby improving imbalanced graph learning performance.

However, directly optimizing the homophily defined in Definition 1 is an optimization problem without constraints on node degrees, which harms GNN performance by incurring message-passing bottlenecks [21, 23]. In response, we design a constrained optimization that maximizes homophily subject to node degree constraints, as shown below.

$$\begin{aligned} \text{maximize} \quad & \mathbb{E}(\mathrm{homo}(G)) = \frac{\sum_i k_i \cdot \mathrm{prob}(v_i, y_i)}{\sum_i k_i}, \\ & \mathrm{prob}(v_i, y_i) = \sum_{y_j \in Y} \mathbb{I}(y_j = y_i) \cdot \frac{S[i,j]}{\sum_k S[i,k]}, \\ \text{s.t.} \quad & k_{\min} \le k_i \le k_{\max} \ \forall i \quad \text{and} \quad \sum_i k_i = N\bar{d}, \end{aligned} \tag{2}$$

where $\mathrm{prob}(v_i, y_i)$ denotes the homophily probability that GoG node v_i connects to neighbors with the same label y_i, based on the similarity matrix S. Here, $\bar{d}$ is the target average GoG node degree,

[2]The implicit complete graph, where edge weights are defined by pairwise similarity [32, 41, 45].

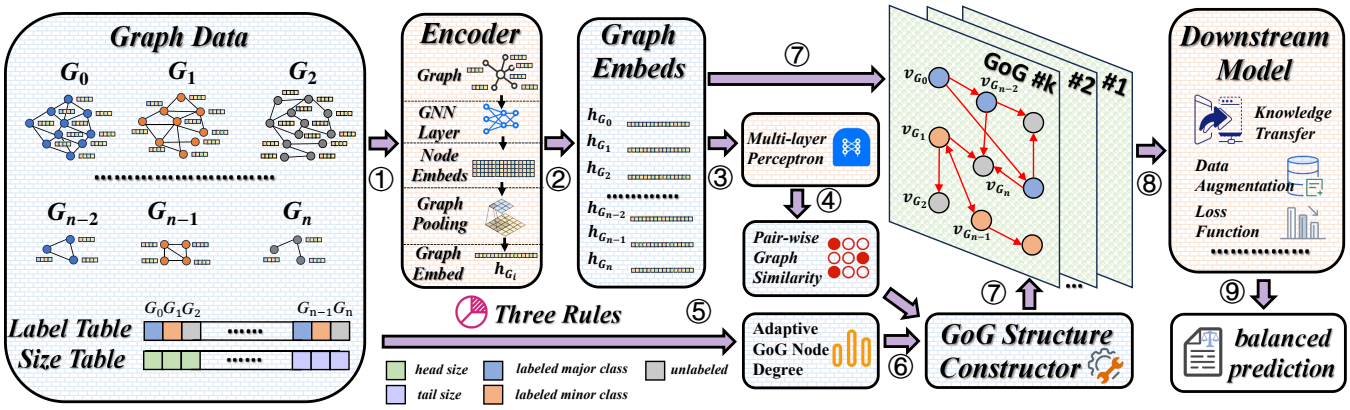


Figure 2: Overview of SamGoG: ① Input graphs to elemental GNN encoders; ② Output graph representations; ③ Use an MLP to compute the classification logits; ④ Compute pair-wise graph similarity matrix; ⑤ Adaptively allocate node degree for GoG nodes by class and size information; ⑥ Construct GoGs by importance sampling based on pair-wise graph similarity and adaptive node degree; ⑦ Combine GoG node features and sampled structure; ⑧ Train node classification model on constructed GoGs; ⑨ Output final result.

which is a pre-defined parameter. $k_{\min}$ and $k_{\max}$ are user-defined bounds that constrain the range of GoG node degrees. N is the number of input graphs, which is also the number of GoG nodes. $\sum_i k_i = N\bar{d}$ ensures that the sum of GoG node degrees is a fixed value, i.e., $N\bar{d}$.

Key Insights. Based on this optimization objective in Equation (2), two critical factors emerge as essential for improving the quality of sampling-based GoG construction. **First**, improving the quality of pairwise graph similarity, i.e., enhancing the similarity among GoG nodes sharing the same label, contributes to a higher value of $\text{prob}(v_i, y_i)$. **Second**, optimizing the distribution of GoG node degrees $\{k_i\}$ can further improve homophily: GoG nodes with higher $\text{prob}(v_i, y_i)$ should be assigned higher degrees k_i to maximize the overall homophily.

This leads us to a fundamental partial ordering principle, which provides a simplified yet effective solution to the optimization problem, as shown in Lemma 1.

Lemma 1. *For any two GoG nodes v_i and v_j with degrees k_i and k_j, if $\mathbb{E}(\text{prob}(v_i, y_i)) > \mathbb{E}(\text{prob}(v_j, y_j))$, then achieving optimal homophily in Equation (2) requires $k_i > k_j$. Conversely, if $\mathbb{E}(\text{prob}(v_i, y_i)) \leq \mathbb{E}(\text{prob}(v_j, y_j))$, then $k_i \leq k_j$ leads to better homophily.*

Lemma 1 is proven in Appendix D [39]. Lemma 1 establishes a relationship between homophily, degree allocation, and expected homophily probability. This principle allows us to prioritize higher degrees to GoG nodes with higher expected $\text{prob}(v_i, y_i)$, thereby improving the overall homophily in the GoG construction process.

3.2 Key Components

3.2.1 Pairwise Graphs-of-Graph Node Similarity Calculation. We first extract the feature representations of GoG nodes by employing a GNN (e.g., GIN [44] and GCN [18]). Through the message passing paradigm of GNNs, these feature representations capture the structural information present in the training graphs. Based on these representations, we further use an MLP-based method to encode the GoG nodes and get initial classification logits ($\text{logits}(g_i)$). To further take into account the label information of the training graphs, we calculate the similarity by incorporating labeled certainty and unlabeled uncertainty, as shown in Equation (3).

$$S[i,j] = P(\hat{y}_i) \cdot P(\hat{y}_j) = \sum_{c \in C} P(\hat{y}_i = c)\, P(\hat{y}_j = c)$$
$$P(\hat{y}_i) = \begin{cases} \text{onehot}(y_i), & g_i \in \mathbb{G}_{\text{L}} \\ \text{Softmax}(\text{logits}(g_i)), & g_i \notin \mathbb{G}_{\text{L}} \end{cases} \tag{3}$$

where $\text{logits}(g_i) \in \mathbb{R}^{|C|}$, $P(\hat{y}_i) \in \mathbb{R}^{|C|}$, and $\sum_{c \in C} P(\hat{y}_i = c) = 1$. Equation (3) adjusts the GoG node similarity by incorporating labeled certainty and unlabeled uncertainty. For labeled graphs ($\{g_i | g_i \in \mathbb{G}_{\text{L}}\}$), we preserve ground-truth certainty through one-hot encoding, while for unlabeled graphs ($\{g_i | g_i \notin \mathbb{G}_{\text{L}}\}$), we maintain probability distributions for the prediction label $\hat{y}_i$. For two graphs g_i and g_j, the joint probability of sharing class c is $P(\hat{y}_i = c, \hat{y}_j = c) = P(\hat{y}_i = c)P(\hat{y}_j = c)$ because of the independent and identically distributed (i.i.d.) assumption as shown in [8, 12, 51]. The full similarity matrix $S \in \mathbb{R}^{N \times N}$ can be computed efficiently as a parallel matrix operation ($S = PP^T$). Finally, S and $\{k_i\}$ can be further used to conduct importance sampling for GoG construction. Compared with methods based on cosine similarity between graph embeddings [36] and kernel-based methods, our method more accurately reflects the probability that a pair of graphs share the same label.

3.2.2 Graphs-of-Graph Node Degree Pre-allocation. Lemma 1 demonstrates that enhancing homophily via degree allocation leads to improved model accuracy. Inspired by this, we propose three rules to allocate GoG node degrees based on these distinct perspectives: the labeled and unlabeled parts: the majority and minority classes in the labeled part; and the head-size and tail-size graphs of the unlabeled part. Based on the constraints in Equation (2), we first assign $k_{\min}$ degrees to each GoG node. The remaining $\Delta = N(\bar{d} - k_{\min})$ degrees are then allocated according to the following rules.

Motivation 1: Labeled and Unlabeled GoG Nodes. First, we design the rule based on the differences between labeled and unlabeled GoG nodes. Since labeled nodes incorporate accurate class information during similarity computation, they tend to achieve better representation than their unlabeled counterparts.

Rule 1 (**Label Priority**). *Given a hyperparameter ratio ρ_1 ($\rho_1 > 1$), we divide Δ degrees to Δ_{L} and Δ_{U} between all the labeled and unlabeled nodes that $\frac{\Delta_{\mathrm{L}}/N_{\mathrm{L}}}{\Delta_{\mathrm{U}}/N_{\mathrm{U}}} = \rho_1$. N_{L} and N_{U} are labeled graph numbers and unlabeled graph numbers, respectively.*

Theorem 1 (Label Priority Justification). *Consider a labeled GoG node $v_i \in \mathbb{G}_{\mathrm{L}}$ and an unlabeled node $v_j \in \mathbb{G}_{\mathrm{U}}$ that share the same ground-truth label y. Under the assumption that prediction confidence on unlabeled data is strictly less than ground-truth certainty (i.e., < 1), we have $\mathbb{E}[\mathrm{prob}(v_i, y)] > \mathbb{E}[\mathrm{prob}(v_j, y)]$ (Proof in Appendix D [39]).*

Parameter Setting (ρ_1). Theorem 1 establishes that labeled nodes possess a higher expected probability of correctness. According to Lemma 1, to maximize homophily, we must assign higher degrees to nodes with higher probabilities (i.e., $k_i > k_j$). This theoretical constraint directly dictates the setting of the hyperparameter ρ_1: it must be set strictly greater than 1 to ensure the average degree of labeled nodes exceeds that of unlabeled ones. In practice, we set $\rho_1 \in \{5, 9\}$ according to the labeled data proportion.

Motivation 2: Major and Minor Classes in the Labeled GoG Nodes. Secondly, we design the rule based on the differences between major and minor classes in the labeled part. We assume that there is no shift in the class distribution between the labeled and unlabeled portions. As shown in Figure 1(a), major classes achieve higher accuracy compared to minor classes, which shows that major classes have better representation. Motivated by this, we design the class imbalance rule as follows.

Rule 2 (**Class Imbalance**). *Given a hyperparameter ratio $\rho_2 > 1$, we divide Δ_{L} degrees to Δ_{major} and Δ_{minor} between all labeled nodes in majority and minority classes, such that $\frac{\Delta_{\mathrm{major}}}{\Delta_{\mathrm{minor}}} = \rho_2$.*

Theorem 2 (Class Imbalance Justification). *Let v_i be a labeled node from the majority class and v_j from the minority class. Assuming the model learns better representations for majority classes, it holds that $\mathbb{E}[\mathrm{prob}(v_i, y_i)] > \mathbb{E}[\mathrm{prob}(v_j, y_j)]$ (Proof in Appendix D [39]).*

Parameter Setting (ρ_2). Similar to Rule 1, Theorem 2 proves the expected probability gap between majority and minority classes. To satisfy the homophily maximization condition in Lemma 1, the degree allocation must favor the majority class. Thus, the hyperparameter ρ_2 is explicitly set to $\rho_2 > 1$, acting as a skewness control factor that aligns the degree distribution with the class-wise prediction confidence. In practice, we set $\rho_2 \in \{1, 3, 9\}$ according to class imbalance ratios. For a higher imbalance ratio, we set a higher ρ_2.

Motivation 3: Size Distributions in the Unlabeled GoG Nodes. Thirdly, we design the rule based on the differences in the size distributions of the unlabeled GoG nodes. As shown in Figure 1(a), the imbalanced graph size leads to imbalanced classification performance. GNNs struggle to generalize across graph sizes not well represented in the training set, particularly when extrapolating from small to big graphs. In scenarios without a significant size distribution shift, head-size graphs typically achieve better performance. ImbGNN [45] highlights that for graphs with significant size differences but similar semantic information and the same true label, graph kernel functions tend to underestimate their similarity, hindering cross-size knowledge transfer in GoG-based methods. To address this, we introduce a window-based strategy, in which higher degrees are assigned to GoG nodes whose sizes better align with the training graph size distribution. The measure of alignment in size is quantified using a size window.

Rule 3 (**Size Adaptation**). *Let $w_{\mathrm{train}} = \{s_i | g_i \in \mathbb{G}_L\}$ denote the size distribution of training graphs and $2r$ be the window width. For each unlabeled graph g_i with size s_i, define its size window $w_i = [s_i - r, s_i + r]$. The allocated degrees Δ_{U} are distributed in proportion with $|w_{\mathrm{train}} \cap w_i|$ as weight, where $|w_{\mathrm{train}} \cap w_i|$ counts training graphs within v_i's size window.*

Theorem 3 (Size Adaptation Justification). *Given two unlabeled graphs $g_i, g_j \in \mathbb{G}_{\mathrm{U}}$ with sizes s_i, s_j and the same true label, if the local training density $|w_{train} \cap w_i| \gg |w_{train} \cap w_j|$, then due to better generalization in dense regions, $\mathbb{E}(\mathrm{prob}(v_i, y_i)) > \mathbb{E}(\mathrm{prob}(v_j, y_j))$ (Proof in Appendix D [39]).*

Parameter Setting (r). The hyperparameter r defines the sensitivity of the size alignment. It determines the window width $2r$ used to calculate the intersection count $|w_{\mathrm{train}} \cap w_i|$. By Theorem 3 and Lemma 1, allocating degrees proportional to this count (effectively setting $k_i > k_j$ for size-aligned nodes) theoretically guarantees improved homophily. In practice, we set $r \in \{2, 20, 50\}$ according to the maximum graph size in different datasets because the overall size distribution varies. Specifically, we set $r = 2$ when the maximum size is less than 100, $r = 20$ for the range $[100, 500]$, and $r = 50$ for sizes exceeding 500. We provide detailed parameter setting analysis in Appendix H [39].

Since the entire GoG node degree allocation process does not require the learned representations, it can be performed offline.

Hierarchical Integration of the Three Rules. The three degree allocation rules are designed to operate in a hierarchical, non-conflicting manner. The allocation proceeds sequentially: (1) Rule 1 first partitions the total degree budget Δ between labeled and unlabeled subsets based on ρ_1, reflecting the fundamental confidence gap between ground-truth labels and predictions; (2) Rule 2 then refines the allocation within the labeled subset by adjusting degrees between majority and minority classes according to ρ_2, addressing class-wise representation quality differences. For labeled nodes in each major (resp. minor) class, we distribute the degrees evenly among them. After allocating the respective totals for the labeled GoG nodes of the major and minor classes, the distribution is then performed equally; (3) Finally, Rule 3 operates exclusively on the unlabeled subset, distributing degrees based on size-window alignment with the training distribution via parameter r, tackling graph size imbalance. This hierarchical design ensures that each rule addresses a distinct aspect of imbalance at its appropriate scope, with no overlap or conflict. The progressive refinement—from coarse-grained labeled/unlabeled partitioning to fine-grained class-wise and size-wise adjustments—allows the three rules to synergistically maximize homophily across multiple dimensions of heterogeneity in the dataset.

Integration with Downstream Models. After constructing the GoG structure via adaptive degree allocation and importance

sampling, SamGoG integrates seamlessly with any inductive semi-supervised node classification model. This training paradigm is analogous to subgraph-based methods such as GraphSAINT [47] and Cluster-GCN [6], but operates at the graph-of-graphs level. Concretely, in each training epoch: (1) a GNN encoder (e.g., GIN or GCN) first computes graph-level representations via message passing and readout, producing GoG node features; (2) the importance sampler constructs a GoG by selecting edges based on the similarity matrix S and allocated degrees $\{k_i\}$; (3) the downstream node classification model (e.g., TailGNN [21]) is then trained on this sampled GoG subgraph for one step, and parameters are updated via backpropagation. By sampling a sequence of multiple GoGs across epochs, this approach reduces inductive bias inherent in one-shot construction while enabling inter-graph message passing. Compared to the fixed k strategy in standard kNN, our dynamic allocation for different GoG nodes can better perceive class and size imbalance, thereby improving graph classification quality.

3.3 Analysis

Sampling Analysis. To establish theoretical guarantees, we analyze the relationship between sample size t (number of GoG constructions) and downstream performance.

Theorem 4. ***Convergence of Sampling***. *Every sampled subgraph-based gradient constitutes an unbiased estimator of the full-graph objective's gradient. Therefore, GCN parameters $\Theta^{(t)}$ converge in expectation to Θ_{full}, satisfying $\lim_{t\to\infty}\mathbb{E}[H^{(t)}] = H_{\text{full}}$. H_{full} denotes node embeddings trained on the complete similarity-weighted graph.*

Theorem 4 is proven in Appendix D [39]. Theorem 4 shows that the performance of the GoG sampling-based method depends on the number of GoG constructions. When t is sufficiently large, our sampling-based method can closely approximate full-graph training, which leverages all available information. However, training on the complete graph, while allowing for greater information utilization, also results in low training efficiency. In contrast, we achieve better efficiency by optimizing importance sampling with adaptive GoG node degree and training in parallel. We additionally provide a comparison with existing sampling methods in Appendix E [39].

Theorem 5. ***Variance Analysis with t.*** *With diminishing learning rates, $\mathrm{Var}(\Theta^{(t)})$ remains bounded with scaling behavior $O(1/t)$. Consequently, $\mathrm{Var}\left(H^{(t)}\right) = O\left(\frac{1}{t}\right)$, causing final embeddings to concentrate around the full-graph solution as t increases.*

Theorem 5 is proven in Appendix [39]. We additionally analyze the sampling based on variance as shown in Theorem 5. This variance-reduction property demonstrates that increasing sample size t reduces inductive bias, consistent with the law of large numbers. Our sampling framework is also highly scalable in practice with GPU-optimized operations. It completes GoG construction for batches of over 1,000 graphs within milliseconds, which is several orders of magnitude faster than computing individual graph embeddings.

Complexity Analysis. Beyond propagation of models, the additional computational overhead primarily stems from three components in GoG construction: pairwise graph similarity calculation, adaptive GoG node degree allocation, and the sampling-based construction of the GoG structure. Here, N represents the number of graphs, and $|C|$ denotes the number of classes. For similarity calculation, we compute similarity via matrix multiplication, with a time complexity of $O(N^2|C|)$, which is significantly lower than graph kernel methods. The GoG node degree allocation algorithm assigns the total number of edges based on predefined rules, resulting in a time complexity of $O(N)$. For sampling-based construction of the GoG structure, its time complexity on a single GoG node is $O(N)$, primarily due to normalization and cumulative distribution function calculations. Since neighborhoods of all N GoG nodes are sampled and aggregated in parallel, the overall time complexity for constructing the GoG remains $O(N)$. In comparison, the overall time complexity of SamGoG is $O(N^2|C|)$, while that of existing GoG methods with kernel-based similarity [41, 45] is $O(N^2|\mathcal{V}|^4)$ [41], and other baseline methods [20, 36, 48, 49] are $O(\sum_{i=1}^{N}(|\mathcal{V}_i|+|\mathcal{E}_i|))$, where $|\mathcal{V}_i|$ and $|\mathcal{E}_i|$ represent the number of nodes and edges in each graph. We additionally provide an analysis of scalability and memory efficiency in Appendix E [39].

4 Experiments

In this section, we conduct extensive experiments to answer the following research questions:

- **(RQ1)** Can SamGoG effectively enhance the overall performance of imbalanced graph classification? Can SamGoG achieve the best performance under varying levels of size and class imbalance? Is SamGoG effective and robust when equipped with different GNN encoders and downstream models?
- **(RQ2)** Can SamGoG help improve the classification performance on minority class and tail-size graphs?
- **(RQ3)** Can SamGoG achieve better edge homophily?
- **(RQ4)** Is SamGoG efficient on large-scale graph data?

4.1 Experiment Setup

Benchmark Settings. We evaluate SamGoG on standard imbalanced classification benchmarks, IGL-Bench [33]. Following IGL-Bench, we adopt TUDatasets [26] and ogbg-molhiv [16] as datasets; details are provided in Appendix C [39]. We use the predefined splits of each dataset with varying class/graph size imbalance ratios (defined in Equations (4) and (5) in Appendix B). Our implementation uses either GIN [44] or GCN [18] as base encoders and employs TailGNN [21] as the downstream model. Other implementation details are provided in Appendix C [39].

Baselines. We compare against all graph classification baselines in IGL-Bench [33], with additional comparisons of our GoG construction against kernel-based methods [4, 15, 30, 34] in G^{2}GNN [41].

4.2 Performance Analysis

To answer **RQ1**, as shown in Table 1, SamGoG achieves state-of-the-art performance on 23/24 for both imbalance types, surpassing the second-best baseline by **5.07%** (class) and **4.13%** (size) in average accuracy. For class imbalance, while baselines vary greatly in performance across datasets, SamGoG consistently delivers high-quality results and remains SOTA, especially under high imbalance. It surpasses the current best baseline, DataDec, by 5.60% on average. Similarly, under size imbalance scenarios, SamGoG retains its

Table 1: Comprehensive performance comparison (% accuracy ± stddev) with changing class or size imbalance levels over 10 run. Red and Blue highlight the best and second-best results respectively.

	GIN Encoder					GCN Encoder				
Algorithm	**PTC-MR**	**D&D**	**IMDB-B**	**REDDIT-B**	**COLLAB**	**PTC-MR**	**D&D**	**IMDB-B**	**REDDIT-B**	**COLLAB**
Panel A: Class Imbalance Scenarios										
Balance ($\rho_{\text{class}} = 5:5$)										
Backbone	50.43±2.69	64.04±3.79	66.05±2.57	76.66±4.80	65.31±3.25	48.62±7.12	63.97±4.09	59.76±1.78	66.65±1.06	61.23±3.38
G^2GNN	53.70±3.87	66.07±2.27	61.91±3.77	72.34±2.76	53.82±2.26	44.35±4.32	61.47±7.70	58.35±3.47	67.02±2.55	49.49±3.18
TopoImb	50.91±2.18	66.57±3.81	66.28±1.85	73.99±1.18	65.92±1.36	50.61±6.39	56.13±4.61	49.52±1.01	58.62±6.52	58.63±1.93
DataDec	54.05±4.85	64.46±1.88	64.09±5.75	**79.29±8.18**	72.24±0.19	54.23±4.16	63.10±2.62	58.20±4.05	69.08±4.49	69.03±1.89
ImGKB	53.48±3.50	65.45±2.88	50.16±0.34	50.24±0.29	39.34±10.88	52.83±5.45	64.93±4.44	50.15±0.39	50.29±0.19	40.44±11.33
SamGoG	**59.71±2.12**	**72.59±1.93**	**69.27±1.53**	77.74±0.50	**72.47±1.58**	**59.49±2.68**	**75.13±0.90**	**71.70±0.79**	**80.05±0.23**	**70.93±1.35**
Low Imbalance ($\rho_{\text{class}} = 7:3$)										
Backbone	47.83±2.95	51.05±5.07	62.31±3.99	61.10±4.86	65.01±1.33	43.84±7.03	58.32±1.51	49.06±2.17	61.85±3.89	53.81±1.88
G^2GNN	51.88±6.23	56.29±7.30	63.87±4.64	69.58±3.59	62.05±3.06	47.86±9.03	65.65±5.41	53.11±3.97	66.02±2.08	54.57±3.30
TopoImb	44.86±3.52	49.97±7.24	59.95±5.19	59.67±7.30	65.88±0.75	45.90±6.41	44.63±4.82	49.75±4.81	54.35±3.13	57.72±1.71
DataDec	55.72±2.88	63.51±1.62	67.92±3.37	78.39±5.01	71.48±1.03	55.86±2.49	64.66±2.04	57.06±5.97	66.45±5.38	71.26±0.91
ImGKB	50.11±5.95	65.85±3.70	47.74±0.29	48.57±2.14	51.21±0.10	49.49±5.12	67.44±3.50	47.65±0.23	48.58±2.15	51.21±0.10
SamGoG	**57.25±2.85**	**73.31±1.99**	**67.96±0.57**	**78.67±1.44**	**72.17±1.58**	**56.98±3.65**	**74.81±0.99**	**68.35±1.59**	**80.03±1.03**	**72.26±1.83**
High Imbalance ($\rho_{\text{class}} = 9:1$)										
Backbone	39.42±1.87	41.54±6.57	53.57±3.21	55.56±7.85	62.00±3.08	40.58±7.61	42.48±2.61	45.09±0.18	54.39±5.16	50.47±0.38
G^2GNN	46.52±9.94	55.38±15.60	59.44±6.49	63.22±4.67	62.61±1.14	41.78±7.61	44.59±4.72	56.40±1.71	65.08±3.08	58.02±3.29
TopoImb	39.42±1.24	39.12±1.62	47.75±3.73	51.58±4.69	64.19±1.77	40.17±2.21	36.50±1.44	46.18±3.06	52.21±3.56	61.40±2.44
DataDec	58.69±3.10	65.77±2.71	66.30±6.70	77.72±5.12	71.50±1.15	55.41±3.37	65.07±2.40	58.58±5.07	65.56±8.61	70.48±0.49
ImGKB	44.24±5.65	59.99±7.57	47.08±3.72	51.25±5.10	50.20±0.06	45.98±8.71	58.21±10.60	46.06±2.74	51.23±5.09	50.20±0.06
SamGoG	**58.81±2.30**	**71.94±1.73**	**66.66±2.25**	**78.41±1.63**	**72.64±1.56**	**59.49±1.26**	**66.90±1.83**	**61.55±1.30**	**76.71±1.30**	**72.18±2.72**
Panel B: Size Imbalance Scenarios										
Low level Size Imbalance										
Ratio (ρ_{size})	*1.60*	*1.56*	*1.64*	*2.79*	*7.94*	*1.60*	*1.56*	*1.64*	*2.79*	*7.94*
Backbone	52.17±4.36	65.01±0.69	66.38±4.27	67.41±2.32	64.34±3.55	51.59±7.07	64.82±0.75	69.15±1.44	53.44±1.78	64.29±1.42
SOLT-GNN	47.54±4.33	63.78±1.06	64.20±5.46	49.57±6.78	61.79±2.09	51.45±3.13	65.69±2.36	68.23±2.48	37.58±3.04	63.82±2.64
TopoImb	49.71±1.98	65.16±1.76	66.00±2.41	69.47±3.70	62.71±2.42	46.74±1.57	56.18±2.10	68.20±0.70	56.94±3.23	60.47±0.91
SamGoG	**53.86±2.12**	**68.22±2.40**	**66.82±0.85**	**72.20±4.24**	**65.01±1.35**	**52.83±0.90**	**71.06±3.54**	**70.53±0.87**	**72.60±1.72**	**65.58±1.35**
Mid level Size Imbalance										
Ratio (ρ_{size})	*2.90*	*2.53*	*2.11*	*4.92*	*11.66*	*2.90*	*2.53*	*2.11*	*4.92*	*11.66*
Backbone	51.38±6.78	61.46±2.43	65.08±5.78	68.32±1.77	65.84±3.12	50.00±5.87	63.99±2.16	67.95±2.82	67.96±0.89	67.64±2.04
SOLT-GNN	53.04±3.91	63.33±1.86	69.38±1.23	73.51±1.14	69.69±2.45	56.23±0.84	62.78±1.33	70.25±1.15	61.95±5.55	66.94±3.67
TopoImb	51.59±4.30	65.99±1.25	68.10±0.87	71.54±0.75	68.68±1.34	54.13±4.62	64.73±7.09	68.75±0.76	69.12±0.52	66.48±1.03
SamGoG	**57.17±1.06**	**71.91±1.22**	**69.53±1.01**	**79.12±0.90**	**70.15±1.42**	**57.03±1.46**	**72.80±2.53**	**70.54±0.40**	**79.08±1.36**	**67.89±1.35**
High level Size Imbalance										
Ratio (ρ_{size})	*4.93*	*8.91*	*4.45*	*10.97*	*16.89*	*4.93*	*8.91*	*4.45*	*10.97*	*16.89*
Backbone	48.41±7.07	60.68±6.89	62.60±3.82	67.41±2.23	67.05±2.46	49.93±5.90	60.98±6.71	64.88±2.02	66.38±0.46	68.99±1.36
SOLT-GNN	51.74±5.25	64.97±3.24	65.03±4.12	60.24±2.11	67.12±3.28	54.78±6.03	63.21±3.40	**69.80±2.07**	67.05±2.54	65.57±5.59
TopoImb	51.96±1.16	64.16±2.96	66.75±0.91	69.14±4.83	67.52±0.77	51.81±1.26	69.66±1.92	66.60±0.91	69.09±1.00	67.74±0.63
SamGoG	**59.13±1.32**	**69.15±1.87**	**67.13±1.59**	**72.31±2.07**	**68.09±1.16**	**59.71±1.46**	**74.65±1.45**	69.38±0.30	**77.29±1.26**	**69.16±1.12**

stability and high performance, achieving a maximum improvement of 15.6% over the baselines. Additionally, SamGoG achieves SOTA results regardless of the backbone used, further validating its robustness.

To answer **RQ2**, balanced accuracy and macro-F1 scores in Appendix F [39], confirm consistent improvements across minority classes.

To further demonstrate SamGoG's effectiveness in addressing class imbalance, we conduct a detailed analysis of recall rates under two class imbalance ratios ($\rho_{\text{class}} = 3:1$ (Mid) and $9:1$ (High)), as shown in Figure 3. The results reveal a striking pattern: while vanilla GIN achieves high recall on the majority classes (e.g., 91.33% on IMDB-BINARY at $\rho_{\text{class}} = 9:1$), its recall on minority classes suffers dramatically, dropping to as low as 27.00% under high imbalance. In contrast, SamGoG substantially improves minority class recall across all scenarios. While SamGoG's majority class recall is slightly lower than vanilla GIN (a trade-off inherent to balancing strategies), it maintains reasonably high performance (e.g., 77.50% on IMDB-BINARY) while dramatically improving minority class recognition.

For graph size imbalance, in Figure 4, SamGoG enhances accuracy on both head- and tail-size graphs compared to GIN.

Figure 5 empirically validates our theoretical analysis of dynamic multi-sampling (Theorems 4 and 5). The figure shows training loss and test accuracy curves as a function of sampling iterations t on the D&D dataset under mid-level class imbalance. Consistent with our convergence theorem ($\mathbb{E}[H^{(t)}] \to H_{\text{full}}$ as $t \to \infty$), test accuracy improves steadily and stabilizes after approximately 10–15 iterations, while training loss decreases monotonically. This demonstrates that multiple GoG constructions effectively reduce the inductive bias of one-shot sampling, allowing the model to converge to a better

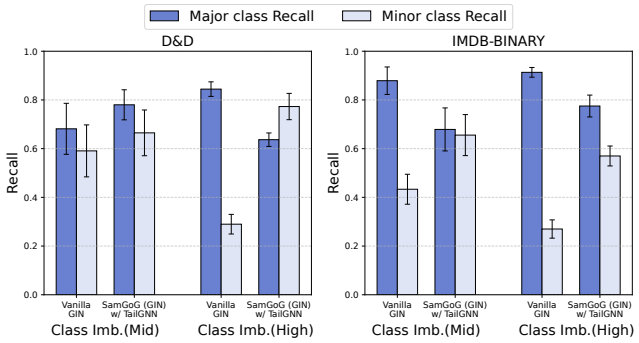


Figure 3: Recall on major/minor class by vanilla GIN and SamGoG (GIN).

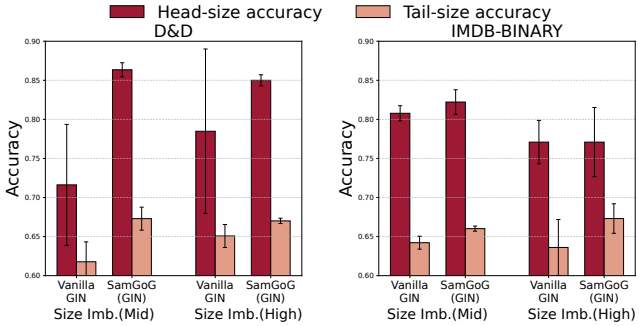


Figure 4: Test accuracy (± stddev) on head-/tail-size graphs by vanilla GIN and SamGoG with GIN as encoder.

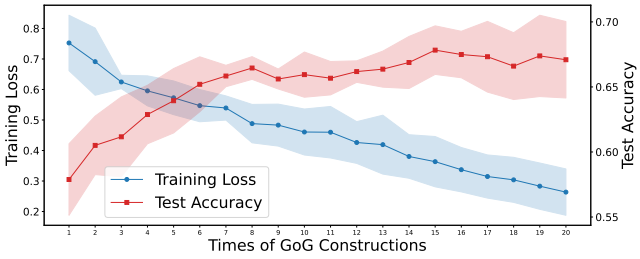


Figure 5: Train loss and test accuracy (± stddev) of SamGoG(GCN) on 20 times of GoG Constructions and downstream model training epochs on mid-level class imbalance split of D&D dataset.

solution. More detailed sensitivity analysis in Appendix F–I [39] further quantifies this effect, showing that performance saturates at $t \approx 100$ with empirically confirmed $O(1/t)$ variance reduction. These results provide strong evidence that our sampling framework is not merely an ensemble technique but a principled approach with rigorous theoretical foundations.

4.3 Ablation Study

To answer **RQ3** and validate our design choices, we first demonstrate that SamGoG constructs GoGs with superior edge homophily. As shown in Figure 6, our learnable similarity metric consistently

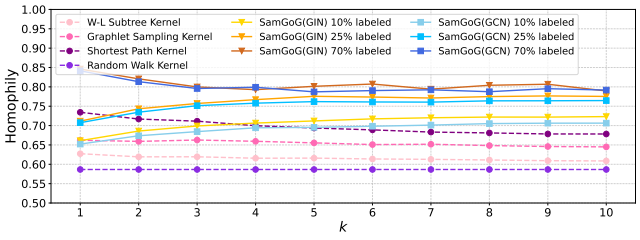


Figure 6: Edge homophily with different average degree k: SamGoG with GIN or GCN as encoder on 10%/25%/70% labeled data preserve competitive edge and node homophily.

produces GoGs with higher homophily compared to various graph kernel methods used by G^2GNN. This improved homophily directly translates to better performance, explaining why SamGoG significantly outperforms G^2GNN in our main experiments Table 1.

Next, we quantify the individual impact of our adaptive node degree allocation rules (Rules 1–3). We create three variants of SamGoG, each removing one rule, and evaluate them on the D&D dataset. Rules 1 (Label Priority) and 2 (Class Imbalance) are tested under class imbalance, while Rule 3 (Size Adaptation) is tested under size imbalance. The results, presented in Figures 7, show that removing any of the rules leads to a noticeable drop in both edge homophily and final classification accuracy. The full model, integrating all three rules, consistently achieves the best results.

4.4 Efficiency Studies

To answer **RQ4**, we measure the time needed for two key steps in the GoG pipeline: calculating the pairwise graph similarity matrix and building the GoGs by sampling. We test this on the TUDataset (about 2,000 graphs) and the ogbg-molhiv dataset (over 41,000 graphs). In Figure 8, SamGoG achieves remarkable efficiency in pairwise similarity computation, completing the task in milliseconds on the TUDataset series (about 1,000~2,000 graphs). This represents a significant improvement over graph kernel methods, which require seconds to minutes for preprocessing at comparable scales. SOLT-GNN [20] requires 2,276 seconds for preprocessing and 500 epochs on REDDIT-B, while SamGoG with an equally-sized model completes in 340 seconds, achieving a 6.7× speedup.

4.5 Dual Imbalance Experiments

Real-world graph datasets often exhibit not only class imbalance but also graph size imbalance simultaneously. To evaluate whether SamGoG can handle their co-occurrence, we construct extreme dual imbalance settings where both ρ_{class} and ρ_{size} are pushed to high values. Specifically, we inject $\rho_{\text{class}} = 9:1$ and $\rho_{\text{size}} = 10.69$ (D&D) or 29.80 (REDDIT-B) on the two datasets. GIN is used as the backbone encoder, and we compare four model variants: (1) Vanilla GIN with no imbalance handling; (2) SamGoG with Rule 2 (class imbalance rule) ablated; (3) SamGoG with Rule 3 (size adaptation rule) ablated; and (4) the full SamGoG model with all three rules active.

Table 2 shows that the full SamGoG model substantially outperforms Vanilla GIN under dual imbalance on both datasets (D&D: 64.93% vs. 58.29%; REDDIT-B: 68.80% vs. 61.20%). Ablating either

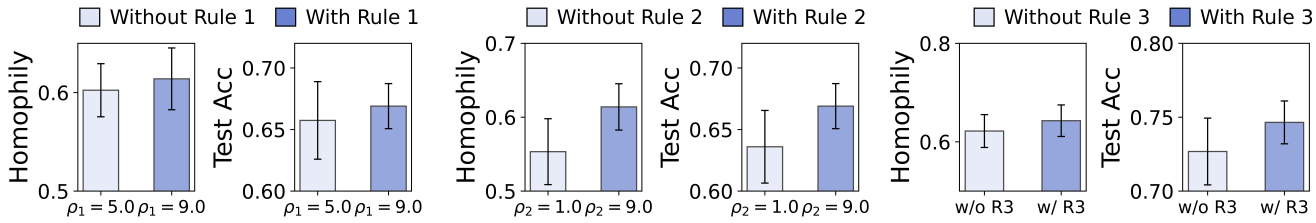


Figure 7: Ablation study: Edge Homophily and test accuracy (± standard deviation) w/o and w/ Rule 1, Rule 2, and Rule 3, respectively.

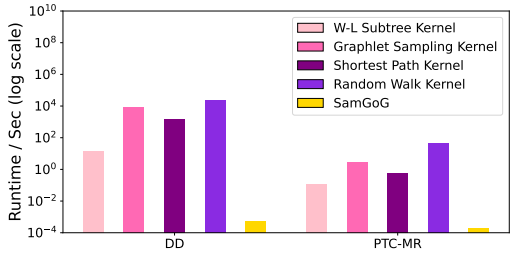


Figure 8: Runtime (seconds) of four kernel methods and SamGoG on D&D and PTC-MR datasets.

Table 2: Results (% ± stddev) under high dual imbalance on D&D and REDDIT-B. Ablating any rule degrades performance, and the full model outperforms all ablated variants.

Dataset	Model	Acc	BAcc	Macro-F1
D&D	Vanilla GIN	58.29 ± 1.04	57.58 ± 0.82	58.72 ± 0.96
	SamGoG (w/o Rule 2)	60.64 ± 0.23	59.27 ± 0.36	60.72 ± 0.46
	SamGoG (w/o Rule 3)	63.05 ± 0.48	62.83 ± 0.72	65.17 ± 0.63
	SamGoG	**64.93 ± 0.18**	**63.90 ± 0.34**	**66.79 ± 0.48**
REDDIT-B	Vanilla GIN	61.20 ± 1.36	61.93 ± 1.12	61.01 ± 1.53
	SamGoG (w/o Rule 2)	64.46 ± 1.26	64.17 ± 0.93	63.63 ± 1.64
	SamGoG (w/o Rule 3)	67.92 ± 1.38	66.25 ± 1.14	67.33 ± 1.36
	SamGoG	**68.80 ± 1.81**	**69.61 ± 1.15**	**68.62 ± 1.85**

Rule 2 or Rule 3 consistently degrades performance, confirming that both rules contribute meaningfully. Crucially, the three rules operate synergistically without mutual interference: Rule 1 partitions the degree distribution between labeled and unlabeled nodes; Rule 2 refines the degree target within the labeled subset; and Rule 3 adapts the degree target for the unlabeled subset. This hierarchical design ensures that each rule addresses a distinct aspect of imbalance at its appropriate scope.

4.6 3D Graph Classification on ModelNet10

Beyond standard 2D graph datasets, we evaluate SamGoG on 3D graph classification to test its generalizability to point cloud data. We use ModelNet10 [43], a benchmark 3D dataset with 10 categories and a class imbalance ratio of $\rho_{class} = 2.3$. Two widely used 3D encoders are adopted: PointNet++ [31] and DGCNN [40]. For each encoder, we compare the vanilla version (encoder only) against SamGoG (encoder + GoG construction) under identical training settings. Evaluation metrics include Accuracy, Balanced Accuracy, Macro-F1, and Minority Class Recall.

Table 3: 3D graph classification results on ModelNet10. SamGoG consistently improves over vanilla backbones, with the largest gain on Minority Class Recall.

Model	Acc (%)	BAcc (%)	Macro-F1 (%)	Minor Recall (%)
Vanilla PointNet++	87.05±0.94	82.63±1.20	78.87±1.83	49.59±2.88
SamGoG (PointNet++)	**88.52±1.23**	**83.73±1.63**	**80.03±1.96**	**54.27±3.24**
Vanilla DGCNN	90.30±0.54	87.37±0.78	83.92±0.90	71.71±2.11
SamGoG (DGCNN)	**91.07±0.67**	**88.12±0.85**	**84.85±1.08**	**75.36±2.73**

Table 3 shows that SamGoG delivers consistent improvements across both 3D encoders. The most notable gain appears on Minority Class Recall (+4.68% for PointNet++, +3.65% for DGCNN), confirming that the GoG mechanism effectively mitigates class imbalance even when the underlying encoder operates on 3D point clouds rather than 2D graphs. This architecture-agnostic behavior stems from the fact that GoG construction operates at the macro-structural level—it does not depend on the specific encoder architecture, whether a GNN for 2D graphs or a point cloud network for 3D data.

5 Conclusion

This work introduces SamGoG, a sampling-based graph-of-graphs framework that effectively tackles class and graph size imbalance in graph classification. SamGoG enhances the edge homophily of GoG by learnable pairwise graph similarity and adaptive GoG node degree allocation. The importance-based sampling mechanism ensures efficient construction of multiple GoGs and seamless integration with various downstream GNNs. SamGoG outperforms existing methods in both accuracy and efficiency.

Acknowledgments

We would like to thank the anonymous reviewers for their valuable feedback and constructive comments. This work was supported by the National Natural Science Foundation of China (NSFC) under Grants 62472400 and 62271465, the National Key R&D Program of China under Grant 2025YFC3408300, and the Suzhou Basic Research Program under Grant SYG202338.

References

[1] Hadi Abdine, Michail Chatzianastasis, Costas Bouyioukos, and Michalis Vazirgiannis. 2024. Prot2Text: Multimodal Protein's Function Generation with GNNs and Transformers. In *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (Eds.). AAAI Press, 10757–10765. doi:10.1609/AAAI.V38I10.28948

[2] Yunsheng Bai, Hao Ding, Yang Qiao, Agustin Marinovic, Ken Gu, Ting Chen, Yizhou Sun, and Wei Wang. 2019. Unsupervised Inductive Graph-Level Representation Learning via Graph-Graph Proximity. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019*. ijcai.org, 1988–1994.

[3] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond Low-frequency Information in Graph Convolutional Networks. *CoRR* abs/2101.00797.

[4] Karsten M. Borgwardt and Hans-Peter Kriegel. 2005. Shortest-Path Kernels on Graphs. In *Proceedings of the 5th IEEE International Conference on Data Mining (ICDM 2005)*. IEEE Computer Society, 74–81.

[5] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic Minority Over-sampling Technique. *J. Artif. Intell. Res.* 16 (2002), 321–357.

[6] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) *(KDD '19)*. 257–266.

[7] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *9th International Conference on Learning Representations, ICLR 2021*. OpenReview.net.

[8] Abouzar Choubineh, David A. Wood, and Zahak Choubineh. 2020. Applying separately cost-sensitive learning and Fisher's discriminant analysis to address the class imbalance problem: A case study involving a virtual gas pipeline SCADA system. *Int. J. Crit. Infrastructure Prot.* 29 (2020), 100357.

[9] Chris Drummond, Robert C Holte, et al. 2003. C4. 5, class imbalance, and cost sensitivity: why under-sampling beats over-sampling. In *Workshop on learning from imbalanced datasets II*, Vol. 11.

[10] Rui Duan, Mingjian Guang, Junli Wang, Chungang Yan, Hongda Qi, Wenkang Su, Can Tian, and Haoran Yang. 2024. Unifying Homophily and Heterophily for Spectral Graph Neural Networks via Triple Filter Ensembles. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems, NeurIPS 2024*.

[11] David Duvenaud, Dougal Maclaurin, Jorge Aguilera-Iparraguirre, Rafael Gómez-Bombarelli, Timothy Hirzel, Alán Aspuru-Guzik, and Ryan P. Adams. 2015. Convolutional Networks on Graphs for Learning Molecular Fingerprints. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015*. 2224–2232.

[12] Charles Elkan. 2001. The Foundations of Cost-Sensitive Learning. In *Proceedings of the Seventeenth International Joint Conference on Artificial Intelligence, IJCAI 2001*. Morgan Kaufmann, 973–978.

[13] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Yihong Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph Neural Networks for Social Recommendation. In *The World Wide Web Conference, WWW*. ACM, 417–426.

[14] Shawn Gu, Meng Jiang, Pietro Hiram Guzzi, and Tijana Milenkovic. 2022. Modeling multi-scale data via a network of networks. *Bioinform.* 38, 9 (2022), 2544–2553.

[15] Tamás Horváth, Thomas Gärtner, and Stefan Wrobel. 2004. Cyclic pattern kernels for predictive graph mining. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2004*. ACM, 158–167.

[16] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems, NeurIPS*.

[17] Kanchan Jha, Sriparna Saha, and Hiteshi Singh. 2022. Prediction of protein–protein interaction using graph neural networks. *Scientific Reports* 12, 1 (2022), 8360.

[18] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR, Conference Track Proceedings*. OpenReview.net.

[19] Derek Lim, Xiuyu Li, Felix Hohne, and Ser-Nam Lim. 2021. New Benchmarks for Learning on Non-Homophilous Graphs. *CoRR* abs/2104.01404 (2021).

[20] Zemin Liu, Qiheng Mao, Chenghao Liu, Yuan Fang, and Jianling Sun. 2022. On Size-Oriented Long-Tailed Graph Classification of Graph Neural Networks. In *WWW '22: The ACM Web Conference 2022*. ACM, 1506–1516.

[21] Zemin Liu, Trung-Kien Nguyen, and Yuan Fang. 2021. Tail-GNN: Tail-Node Graph Neural Networks. In *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 1109–1119.

[22] Zhining Liu, Ruizhong Qiu, Zhichen Zeng, Hyunsik Yoo, David Zhou, Zhe Xu, Yada Zhu, Kommy Weldemariam, Jingrui He, and Hanghang Tong. 2024. Class-Imbalanced Graph Learning without Class Rebalancing. In *Forty-first International Conference on Machine Learning, ICML 2024*. OpenReview.net.

[23] Zemin Liu, Wentao Zhang, Yuan Fang, Xinming Zhang, and Steven C. H. Hoi. 2020. Towards Locality-Aware Meta-Learning of Tail Node Embeddings on Networks. In *CIKM '20: The 29th ACM International Conference on Information and Knowledge Management, 2020*. ACM, 975–984.

[24] Sitao Luan, Chenqing Hua, Minkai Xu, Qincheng Lu, Jiaqi Zhu, Xiao-Wen Chang, Jie Fu, Jure Leskovec, and Doina Precup. 2023. When Do Graph Neural Networks Help with Node Classification? Investigating the Homophily Principle on Node Distinguishability. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems, NeurIPS 2023*.

[25] Yao Ma, Xiaorui Liu, Neil Shah, and Jiliang Tang. 2021. Is Homophily a Necessity for Graph Neural Networks? *CoRR* abs/2106.06134.

[26] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. *CoRR* abs/2007.08663 (2020).

[27] Jingchao Ni, Hanghang Tong, Wei Fan, and Xiang Zhang. 2014. Inside the atoms: ranking on a network of networks. In *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '14, 2014*. ACM, 1356–1365.

[28] Shirui Pan and Xingquan Zhu. 2013. Graph Classification with Imbalanced Class Distributions and Noise. In *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence*. IJCAI/AAAI, 1586–1592.

[29] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-GCN: Geometric Graph Convolutional Networks. In *8th International Conference on Learning Representations, ICLR 2020*. OpenReview.net.

[30] Natasa Przulj. 2007. Biological network comparison using graphlet degree distribution. *Bioinform.* 23, 2 (2007), 177–183.

[31] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. 2017. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 5099–5108. https://proceedings.neurips.cc/paper/2017/hash/d8bf84be3800d12f74d8b05e9b89836f-Abstract.html

[32] Jiawen Qin, Pengfeng Huang, Qingyun Sun, Cheng Ji, Xingcheng Fu, and Jianxin Li. 2025. Graph Size-imbalanced Learning with Energy-guided Structural Smoothing. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining, WSDM 2025*. ACM, 457–465.

[33] Jiawen Qin, Haonan Yuan, Qingyun Sun, Lyujin Xu, Jiaqi Yuan, Pengfeng Huang, Zhaonan Wang, Xingcheng Fu, Hao Peng, Jianxin Li, and Philip S. Yu. 2025. IGL-Bench: Establishing the Comprehensive Benchmark for Imbalanced Graph Learning. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*. OpenReview.net.

[34] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. 2011. Weisfeiler-Lehman Graph Kernels. *J. Mach. Learn. Res.* 12 (2011), 2539–2561.

[35] Jeffrey J. Sutherland, Lee A. O'Brien, and Donald F. Weaver. 2003. Spline-Fitting with a Genetic Algorithm: A Method for Developing Classification Structure-Activity Relationships. *J. Chem. Inf. Comput. Sci.* 43, 6 (2003), 1906–1915.

[36] Hui Tang and Xun Liang. 2023. Where to Find Fascinating Inter-Graph Supervision: Imbalanced Graph Classification with Kernel Information Bottleneck. In *Proceedings of the 31st ACM International Conference on Multimedia, MM 2023*. ACM, 3240–3249.

[37] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *6th International Conference on Learning Representations, ICLR, Conference Track Proceedings*. OpenReview.net.

[38] Hanchen Wang, Defu Lian, Ying Zhang, Lu Qin, and Xuemin Lin. 2020. GoGNN: Graph of Graphs Neural Network for Predicting Structured Entity Interactions. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*. ijcai.org, 1317–1323.

[39] Shangyou Wang, Zezhong Ding, and Xike Xie. 2026. SamGoG: A Sampling-Based Graph-of-Graphs Framework for Imbalanced Graph Classification [Appendix]. https://github.com/USTC-DataDarknessLab/SamGoG/blob/main/appendix.pdf.

[40] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. 2019. Dynamic Graph CNN for Learning on Point Clouds. *ACM Trans. Graph.* 38, 5 (2019), 146:1–146:12. doi:10.1145/3326362

[41] Yu Wang, Yuying Zhao, Neil Shah, and Tyler Derr. 2022. Imbalanced Graph Classification via Graph-of-Graph Neural Networks. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, 2022*. ACM, 2067–2076.

[42] Lirong Wu, Haitao Lin, Zhangyang Gao, Cheng Tan, and Stan Z. Li. 2021. GraphMixup: Improving Class-Imbalanced Node Classification on Graphs by Self-supervised Context Prediction. *CoRR* abs/2106.11133.

[43] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3D ShapeNets: A deep representation for volumetric shapes. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*. IEEE Computer Society, 1912–1920. doi:10.1109/CVPR.2015.7298801

[44] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks?. In *7th International Conference on Learning Representations, ICLR, Conference Track Proceedings*. OpenReview.net.

[45] Wei Xu, Pengkun Wang, Zhe Zhao, Binwu Wang, Xu Wang, and Yang Wang. 2024. When Imbalance Meets Imbalance: Structure-driven Learning for Imbalanced Graph Classification. In *Proceedings of the ACM on Web Conference, WWW 2024*. ACM, 905–913.

[46] Zhe Xu, Yuzhong Chen, Qinghai Zhou, Yuhang Wu, Menghai Pan, Hao Yang, and Hanghang Tong. 2023. Node Classification Beyond Homophily: Towards a General Solution. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023*. ACM, 2862–2873.

[47] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2020. GraphSAINT: Graph Sampling Based Inductive Learning Method. In *International Conference on Learning Representations*.

[48] Chunhui Zhang, Chao Huang, Yijun Tian, Qianlong Wen, Zhongyu Ouyang, Youhuan Li, Yanfang Ye, and Chuxu Zhang. 2023. When Sparsity Meets Contrastive Models: Less Graph Data Can Bring Better Class-Balanced Representations. In *International Conference on Machine Learning, ICML 2023 (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 41133–41150.

[49] Tianxiang Zhao, Dongsheng Luo, Xiang Zhang, and Suhang Wang. 2022. TopoImb: Toward Topology-Level Imbalance in Learning From Graphs. In *Learning on Graphs Conference, LoG 2022 (Proceedings of Machine Learning Research, Vol. 198)*. PMLR, 37.

[50] Yilun Zheng, Sitao Luan, and Lihui Chen. 2024. What Is Missing For Graph Homophily? Disentangling Graph Homophily For Graph Neural Networks. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems, NeurIPS 2024*.

[51] Zhi-Hua Zhou and Xu-Ying Liu. 2006. Training Cost-Sensitive Neural Networks with Methods Addressing the Class Imbalance Problem. *IEEE Trans. Knowl. Data Eng.* 18, 1 (2006), 63–77.

[52] Jiong Zhu, Ryan A. Rossi, Anup Rao, Tung Mai, Nedim Lipka, Nesreen K. Ahmed, and Danai Koutra. 2021. Graph Neural Networks with Heterophily. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021*. AAAI Press, 11168–11176.

[53] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems, NeurIPS 2020*.

A Notations

Table 4 provides a comprehensive summary of the mathematical symbols and notations used throughout the paper. Readers may refer to this table as a quick reference when following the technical exposition of the SamGoG framework.

Table 4: Notation table.

Symbol	Definition
$\mathcal{G}, N$	Set of N input graphs ($\mathcal{G} = \{g_i\}_{i=1}^{N}$).
$\mathcal{G}_L$, $\mathcal{G}_U$	Labeled and unlabeled subsets of input graphs.
$g_i = (A_i, X_i, y_i)$	Input graph with adjacency A_i, features X_i, and label y_i.
s_i	Size (#nodes) of input graph g_i.
$\mathbf{A}$, $\mathbf{V}$, $\mathbf{E}$, $\mathbf{H}$, $\mathbf{Y}$	The GoG $\mathbf{A} = (\mathbf{V}, \mathbf{E}, \mathbf{H}, \mathbf{Y})$ and its components: node set $\mathbf{V}$, edge set $\mathbf{E}$, node features $\mathbf{H}$, labels $\mathbf{Y}$.
v_i, h_i	GoG node corresponding to g_i, and its feature vector.
C	Set of possible graph class labels.
$\mathbf{S} \in \mathbb{R}^{N \times N}$	Pairwise graph similarity matrix.
$P(\hat{y}_i)$	Predicted label distribution for g_i (used in JS divergence).
$\text{prob}(v_i, y_i)$	Homophily probability: v_i connects to neighbors with same label y_i.
k_i	Allocated node degree for GoG node v_i.
$\bar{d}$	Target average degree for all GoG nodes.
$k_{\min}$, $k_{\max}$	Minimum and maximum bounds for GoG node degrees.
Δ	Remaining total degrees to allocate ($\Delta = N(\bar{d} - k_{\min})$).
ρ_1	Ratio for degree allocation between labeled & unlabeled nodes (Rule 1).
Δ_L, Δ_U	Degrees allocated to labeled and unlabeled GoG subsets by Rule 1.
ρ_2	Ratio for degree allocation between majority & minority classes (Rule 2).
Δ_{major}, Δ_{minor}	Degrees allocated to majority & minority labeled GoG nodes by Rule 2.
r	Radius of the size window for size-aware degree allocation (Rule 3).
w_i, w_{train}	Size window of g_i ($[s_i - r,\ s_i + r]$) and the training set's size distribution (Rule 3).
t	Number of sequential GoG sampling iterations (multi-shot construction).

B Imbalance Formulations

B.1 Class Distribution Imbalance

Given the labeled training set $\mathbb{G}_L = \{g_i\}_{i=1}^{N_L}$ where each graph g_i belongs to class $y_i \in C$, we define the class imbalance ratio as:

$$\rho_{\text{class}} = \frac{\max_{c \in C} |\{g_i \in \mathbb{G}_L | y_i = c\}|}{\min_{c \in C} |\{g_i \in \mathbb{G}_L | y_i = c\}|} \tag{4}$$

This formulation captures the disparity between the most and least frequent classes in the training data. In practice, such an imbalance leads to biased classifiers that favor majority classes, typically manifesting as low recall rates for minority classes, where predictions disproportionately favor the dominant categories.

B.2 Graph Size Imbalance

For the complete graph dataset $\mathbb{G} = \{g_i\}_{i=1}^{N}$ where each graph g_i contains s_i nodes, we partition $\mathbb{G}$ into head ($\mathcal{H}_g$) and tail ($\mathcal{T}_g$) sets containing the largest 20% and remaining 80% of graphs by size respectively. The size imbalance ratio is:

$$\rho_{\text{size}} = \frac{\frac{1}{|\mathcal{H}_g|} \sum_{\mathcal{G}_i \in \mathcal{H}_g} s_i}{\frac{1}{|\mathcal{T}_g|} \sum_{\mathcal{G}_j \in \mathcal{T}_g} s_j} \tag{5}$$

This structural imbalance creates two distinct challenges: (1) When training and test distributions are aligned, models underperform on tail-size graphs due to insufficient representation; (2) Under distribution shift, models exhibit degraded generalization when extrapolating to size regions sparsely represented in training.

Extended Version Summary

Due to space constraints, this camera-ready version presents the core methodology, theoretical analysis, and main experimental results. A comprehensive extended version of this paper, which includes additional appendices with full technical details, is publicly available at [39].

The extended version contains the following supplementary materials that complement the results presented in this paper.

Appendix C: Dataset Descriptions and Experimental Details. We provide complete descriptions of all benchmark datasets used in our evaluation, including PTC-MR, D&D, IMDB-BINARY, REDDIT-BINARY, COLLAB, and ogbg-molhiv, along with their licensing information and preprocessing procedures. We also report the full computational environment specification, comprehensive hyperparameter ranges for all components (general settings, GoG

construction, and downstream models), and the edge homophily analysis protocol.

Appendix D: Proofs of Theorems. We present rigorous proofs for all theoretical results stated in the main text: (1) the proof of Lemma 1, which establishes the relationship between node degree allocation and edge homophily optimization via the rearrangement inequality; (2) the justification proofs for Rules 1–3 governing adaptive GoG node degree allocation, including the label priority theorem, the class imbalance adaptation, and the size adaptation rule under the size generalization assumption; and (3) the complete proof of Theorems 4 and 5, which establish the convergence and variance reduction guarantees of our sampling-based GoG training procedure.

Appendix E: Extended Discussion on SamGoG. We provide an in-depth comparison between SamGoG's dynamic GoG sampling and existing sampling paradigms such as bootstrap sampling and bagging ensembles, highlighting the fundamental differences in mechanism and theoretical guarantees. We also present a detailed scalability and memory efficiency analysis, including strategies for reducing the $O(N^2)$ memory footprint of the similarity matrix through on-the-fly computation and mini-batch processing.

Appendix F: Additional Results on IGL-Bench. We report the complete balanced accuracy, macro-F1, and minority class recall results for all class imbalance and graph size imbalance experiments across all datasets and both GIN and GCN encoders, complementing the accuracy results presented in Table 1 of the main text.

Appendices G–I: Additional Experiments. The extended version includes further empirical studies evaluating SamGoG's robustness, parameter sensitivity, and scalability. These experiments cover additional ablation analyses on key design choices and model components, sensitivity studies on all major hyperparameters with stability validation, and systematic scalability evaluations across datasets of varying sizes, demonstrating SamGoG's efficiency in both runtime and memory consumption.

Appendix J: Limitations and Broader Impacts. We discuss potential limitations regarding fairness and bias amplification, privacy and data leakage concerns in GoG construction, and efficiency considerations for future work. We also discuss the broader societal impacts of improved imbalanced graph classification in domains such as social network analysis, molecular discovery, and bioinformatics, along with recommendations for responsible deployment.

We encourage readers to consult the extended version for the complete technical exposition and empirical evidence supporting the claims made in this paper.